Student AI – Dynamic Service-Oriented Backend Architecture

1. Introduction

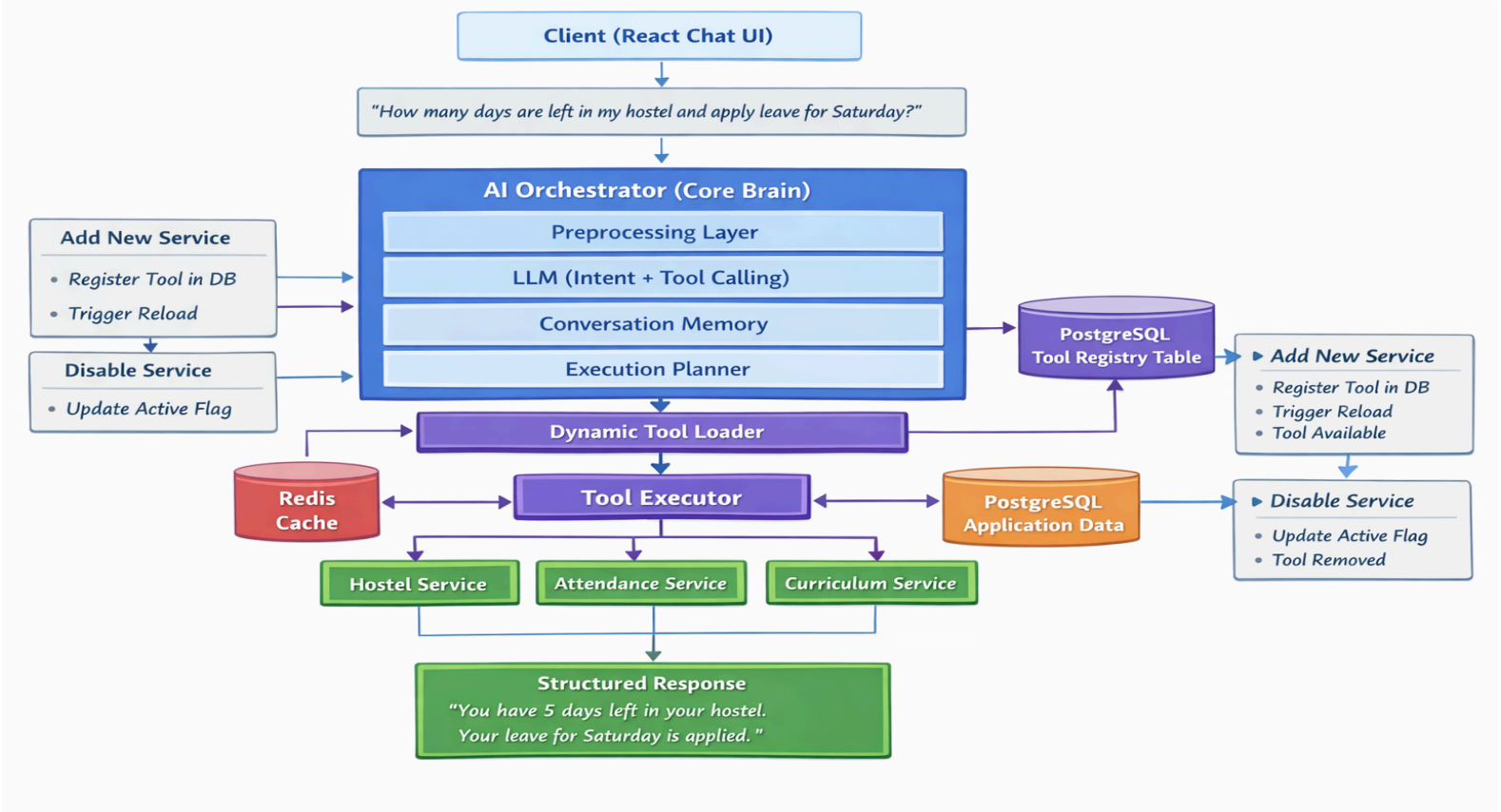
This system is an AI-native backend architecture designed to power a conversational university assistant. It enables users to interact with services like hostels, attendance, and curriculum using natural language.

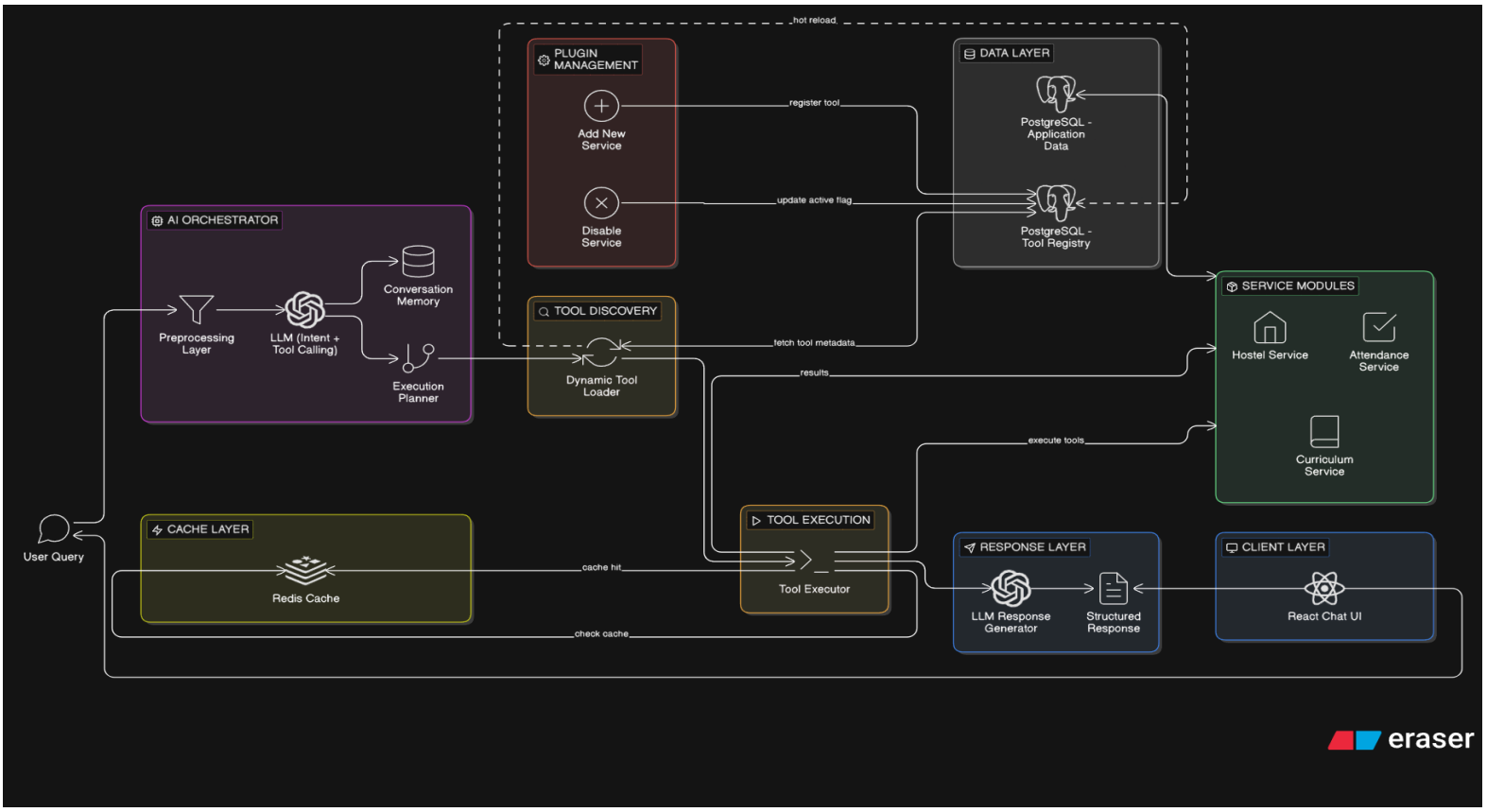
The key innovation is:

Dynamic service plug-and-play using a database-driven tool registry

This allows:

- Adding new services without modifying core code
- Disabling services instantly
- Scaling backend capabilities without redeployment





2. Core Architectural Idea

Instead of hardcoding APIs:

User Query → LLM → Tool → Service

We introduce:

User Query → LLM → Tool Name → DB Tool Registry → Dynamic Loader → Service

This decouples:

- AI logic
- Tool mapping
- Service execution

3. Architecture Breakdown

3.1 AI Orchestrator (Core Brain)

Components:

- Preprocessing Layer
- LLM (Intent + Tool Calling)
- Conversation Memory
- Execution Planner

Role:

- Understand user query
- Convert into tool calls

3.2 Tool Discovery Layer

Component:

Dynamic Tool Loader

Role:

- Fetch tool metadata from DB
- Dynamically import service functions
- Maintain in-memory registry

3.3 Tool Registry (PostgreSQL)

Table: tool_registry

Stores:

- tool name
- module path
- function name
- description
- active flag

Enables:

- Plugging new services
- Disabling services dynamically

3.4 Tool Execution Layer

Component:

Tool Executor

Role:

- Executes selected tools
- Supports multiple tool calls
- Interacts with cache + services

3.5 Cache Layer (Redis)

Role:

- Store frequently accessed results
- Reduce DB load

Example:

- hostel days remaining
- attendance %

3.6 Service Layer (Pluggable)

Examples:

- Hostel Service
- Attendance Service
- Curriculum Service

Key Property:

Completely independent of orchestrator

3.7 Data Layer (PostgreSQL)

Stores:

- student data
- hostel allocation
- attendance
- tool registry

3.8 Response Layer

Component:

LLM Response Generator

Role:

- Convert structured output → natural language
- Combine multi-tool results

4. End-to-End Flow (Live Example)

User Query

“How many days are left in my hostel and apply for Saturday?”

Step 1: LLM Output

```
[
 {"tool": "hostel.get_remaining_days"},
 {
   "tool": "hostel.apply_leave",
   "arguments": {
     "start_date": "2026-04-11",
     "end_date": "2026-04-11"
   }
 }
]
```

Step 2: Planner

Adds:

```
{
 "register_no": "2547140"
}
```

Step 3: Tool Discovery

```
SELECT * FROM tool_registry WHERE active = TRUE;
```

Step 4: Dynamic Loader

```
import importlib

def load_tools_from_db(rows):
  registry = {}
  for tool in rows:
      module = importlib.import_module(tool["module"])
      func = getattr(module, tool["function_name"])
      registry[tool["name"]] = func
  return registry
```

Step 5: Tool Execution

```
def execute(tool_calls, user, registry):
  results = []
  for call in tool_calls:
```

```
    fn = registry[call["tool"]]
    args = call.get("arguments", {})
    args["register_no"] = user["register_no"]
    results.append(fn(**args))
  return results
```

Step 6: Service Execution

```
# services/hostel_service.py

def get_remaining_days(register_no):
  return {"days_left": 120}

def apply_leave(register_no, start_date, end_date):
  return {"status": "pending"}
```

Step 7: Response

“You have 120 days left in your hostel. Your leave request has been submitted.”

5. Dynamic Service Plug-In (LIVE EXAMPLE)

Add New Service (Curriculum)

Step 1: Create Service

```
# services/curriculum_service.py

def get_courses(register_no):
  return ["ML", "DBMS", "AI"]
```

Step 2: Register Tool (NO CODE CHANGE)

```
INSERT INTO tool_registry (
  name,
  module,
  function_name,
  description,
  active
)
VALUES (
  'curriculum.get_courses',
  'services.curriculum_service',
  'get_courses',
  'Get courses for current semester',
  TRUE
);
```

Step 3: Reload Tools

```
POST /reload-tools
```

Result

User:

“What subjects do I have?”

LLM:

```
{
 "tool": "curriculum.get_courses"
}
```

No code changes required

6. Dynamic Service Removal

Disable tool:

```
UPDATE tool_registry
SET active = FALSE
WHERE name = 'hostel.apply_leave';
```

Reload

Result:

User:

“Apply leave”

Response:

“This feature is not available.”

7. Why This Architecture is Powerful

1. True Plug-and-Play

- Add/remove services without touching backend core

2. LLM-driven routing

- No manual routing logic

3. Scalable

- Add 10 → 100 services easily

4. Decoupled

5. Fast iteration

- Only DB change required

8. Limitations

Service code still needed

You must still write:

```
def get_courses():
```

No validation in LLM

Planner must handle:

- missing inputs
- invalid cases

9. Summary

```
User Query
  ↓
LLM (tool selection)
  ↓
Tool Registry (DB)
  ↓
Dynamic Loader
  ↓
Service Execution
  ↓
LLM Response
```

Step 1: Capability Design (Exams Service)

Before implementation, define capabilities (not endpoints)

Step 2: Service Implementation

Create the service module:

```
# services/exam_service.py

def get_next_exam(register_no):
    return {
        "course": "Machine Learning",
        "date": "2026-05-10"
```

```
    }

def get_exam_schedule(register_no):
    return [
        {"course": "ML", "date": "2026-05-10"},
        {"course": "DBMS", "date": "2026-05-12"}
    ]
```

Step 3: Tool Registration (Database)

Insert tool metadata into tool_registry:

```
INSERT INTO tool_registry (name, module, function_name, description, active)
VALUES
('exam.get_next_exam', 'services.exam_service', 'get_next_exam',
 'Get the next upcoming exam for the student', TRUE),

('exam.get_schedule', 'services.exam_service', 'get_exam_schedule',
 'Get full exam schedule for the student', TRUE);
```

Step 4: Dynamic Tool Loading

Trigger reload:

```
POST /reload-tools
```

Runtime Behavior:

```
import importlib

module = importlib.import_module("services.exam_service")
func = getattr(module, "get_next_exam")

TOOL_REGISTRY["exam.get_next_exam"] = func
```

Step 5: LLM Tool Awareness

The system fetches active tools:

```
SELECT name, description FROM tool_registry WHERE active=TRUE;
```

Tools Available to LLM

```
[
  "hostel.get_remaining_days",
  "attendance.get_percentage",
  "exam.get_next_exam",
  "exam.get_schedule"
]
```

Step 6: Intent Detection

User Query

“When is my next exam?”

LLM Mapping

- “exam” → domain
- “next” → upcoming

Output

```
{
  "tool": "exam.get_next_exam",
  "arguments": {}
}
```

Step 7: Execution Planning

Planner performs:

- Injects system parameters
- Validates arguments
- Prepares execution

```
{
  "register_no": "2547140"
}
```

Step 8: Tool Execution

```
fn = TOOL_REGISTRY["exam.get_next_exam"]

result = fn(register_no="2547140")
```

Output

```
{
  "course": "Machine Learning",
  "date": "2026-05-10"
}
```

Step 9: Response Generation

LLM converts structured output:

“Your next exam is Machine Learning on May 10, 2026.”